Lab 1: Environment Setup and First Contact, the Attack Surface of a Machine

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

In this first lab you will install and boot the course lab virtual machine, then look at it the way both an attacker and a defender do: as a collection of things that *listen*, *run with privilege*, *act on a schedule*, and *hold secrets*. You will not exploit anything today. The single skill this lab builds is **enumeration**, systematically discovering and recording a machine's attack surface, and the notes you produce become the raw material for the threat model you build in Lab 2. The work is deliberately observational and tool-light: the enumeration commands in Parts A to E only *read* system state, and nothing assumes prior systems-security background. (The one-off setup steps, creating a working directory, the `hello-world` container check, are the exception, and are marked as such.)

Everything runs inside the one course VM on your own laptop. No external network target is involved, and no privilege is required beyond the ordinary `student` account and `sudo`.

Contents

- Learning Objectives
- Before the lab (at home, ~30 min)
- A note on method
- Part A: Orientation, what is this machine?
- Part B: What listens? The network-facing surface
- Part C: What runs with privilege? Setuid binaries and sudo
- Part D: What acts on a schedule? Cron and writable paths
- Part E: What is exposed at rest? File permissions and secrets
- Part F: Consolidate the attack surface inventory
- Reflection
- References
- Deliverables

Learning Objectives

By completing this lab, students will be able to:

1. Boot and snapshot the course lab VM, installed beforehand in Week 0, and describe the boundary between guest and host.
2. Enumerate the four dimensions of a machine's local attack surface, listening services, privileged binaries, scheduled tasks, and sensitive files, using standard Linux tools.
3. Distinguish an *observation* (what the system exposes) from a *judgement* (whether that exposure is a weakness), and defer the second where the mechanism has not yet been taught.
4. Record an attack surface as a structured, reproducible inventory suitable for threat modelling.

These map to FUC outcomes LO1 (the nature and duality of systems security), LO2 (assets, actors, threats), and LO4 (recognising the security posture of a system).

Before the lab (at home, ~30 min)

Do everything in this section *before* the session, so you arrive with the VM booted and working. The Parts that follow are the in-session, hands-on work.

1. Install the VM

Follow the **Setup Wizard** distributed with this lab. The image ships as a **ready-to-run bundle**, already configured, so you do not build a VM or touch any hypervisor settings:

- **Apple Silicon Mac (M1 to M4):** download `ssc-lab-arm64-v1.0.1.utm.zip` and check its published SHA-256, install **UTM 4.7.5** (the free download from getutm.app), then expand the ZIP (Safari may have expanded it already) and **double-click the `ssc-lab-arm64-v1.0.1.utm` bundle inside** to add it to UTM and press play. The bundle is pre-set to QEMU, ARM64, 4 GB RAM, 2 vCPU, network in **Emulated** mode. On an Apple Silicon Mac an ARM64 guest runs accelerated despite the "Emulate" label. Do **not** use UTM 5 (the v5 line is GPU-only beta and fixes nothing here); stay on 4.7.5.
- **Intel or AMD laptop (Windows, Linux, Intel Mac):** download `ssc-lab-amd64-v1.0.ova` and import it into **VirtualBox** (*File* → *Import Appliance*), then start it. 4 GB RAM, 20 GB disk are preset.

Both images are identical inside: Ubuntu 24.04 LTS, one user `student` with password `student`, in the `sudo` and `docker` groups. The VM boots to a normal `login:` prompt on screen. **4 GB RAM** is enough; raise to 6 to 8 GB only if the Lab 4 identity provider or the capstone stack feels slow. The disk is **20 GB**, baked into the image. **Do not enable a Shared Directory:** this VM runs exploits and is meant to stay isolated from your host. Work from your own terminal over SSH, as the Setup Wizard shows (`ssh -p 2222 student@localhost`): the VM's own window has no shared clipboard, so that is how text gets copied *out*. Move files *in* by downloading them from the course platform inside the VM, or with `scp -P 2222`.

If the UTM display shows "Display output is not active": you have an older build of the bundle with the serial console. The current bundle boots to a graphical `login:`. If you hit the blank display, switch the UTM window from Display to the built-in Serial terminal, or `ssh -p 2222 student@localhost` via the bundle's port-forward. Report it, because the shipped bundle should not do this.

If networking does not work (`ping` or `docker pull` fails): confirm the VM's network mode is **Emulated (VLAN)**, not Shared. The shipped bundle is already Emulated; if yours is not, a stray older copy is the likely cause.

Boot the VM and log in. Confirm the base tooling is present:

```
cat /etc/ssc-image-release      # should name the SSC lab image and your
↪ architecture
docker run --rm hello-world     # confirms Docker works without sudo
```

2. Snapshot before you start

Every lab in this course tells you when to snapshot. Some later labs deliberately break the machine; a snapshot lets you return to a known-good state in seconds. Take one now and name it clearly:

- **UTM:** with the VM **stopped** (right-click the VM in the list to stop it cleanly), use the snapshot control; name it `lab1-clean`. (UTM snapshots the qcow2; taking it while stopped is the reliable path.)
- **VirtualBox:** *Machine* → *Take Snapshot…*, name it `lab1-clean`.

Rule for the whole course: snapshot *before* a lab, restore *after* if anything is left in a bad state. Get into the habit now, while the stakes are zero.

3. Create a working directory for your notes

```
mkdir -p ~/ssc-lab1 && cd ~/ssc-lab1
```

You will paste command output into a notes file as you go. Keep it: Lab 2 builds directly on it.

Preparation questions (submit with Lab report 1, produced in Lab 2)

- **P1.** Before the session, run `date; cat /etc/ssc-image-release` and take a screen capture of the result, then state your VM's architecture. Put the capture in the report as an image, not as pasted

text: the date in it is your proof that the machine was running before the lab, which the marking asks for.

- **P2.** In one sentence: what does it mean to *enumerate* a machine's attack surface, and why do it before any exploitation?

A note on method

Throughout this lab you will **observe**, then **explain**, and, where the course has already given you the tools, **judge**. Today you will mostly observe and explain. Several things you find (a setuid binary, a world-readable configuration file) are not yet things you can properly evaluate, because the mechanisms behind them are taught in Blocks 2 and 3. Where that is the case, the lab says so explicitly and asks you only to *record* the fact and *form a question* about it. Resisting the urge to jump to "this is a vulnerability" is itself part of the discipline: an attack surface is enumerated before it is judged.

Part A: Orientation, what is this machine?

Before enumerating the surface, establish the ground truth of the system you are on.

Exercise 1: Identity and baseline. Run each command, record the output in your notes, and answer the questions.

```
whoami; id                      # who am I, and what groups do I belong to?
uname -a                        # kernel, architecture
cat /etc/os-release             # distribution and version
hostnamectl                     # a one-screen summary
```

1a. Which groups is `student` a member of, and which one of them is the most security-relevant? (You do not need to explain *why* yet, just name it and note that you will return to it.)

1b. The kernel version is shared by every process on this machine, including every container you will later run. In one sentence, and in your own words so far, what does "shared kernel" suggest about how strong the boundary between two containers on this host can be? (This is a first intuition, not a definition; Block 4 makes it precise.)

1c. This machine is a guest: everything you break today is inside it. In one sentence, say what the hypervisor keeps apart, and why a shared folder with your own computer would be an exception to it, which is why this unit forbids one.

Part B: What listens? The network-facing surface

A service that accepts connections is a way *in*. The first question an attacker asks of a machine is "what is listening, and as whom?".

How to read a service. The services that start with this machine are started and supervised by **systemd**, and each one is described by a **unit file**: a short text file that says which program runs (`ExecStart=`), as which user (`User=`, or `root` when there is no such line), and when. `systemctl status` says whether it is running now, `systemctl cat` shows the unit file, and `journalctl -u` shows its log; you met all three in Week 0, Part F. They matter here because a listening port only tells you that *something* answers. The unit file is where you find out what it is, whom it runs as, and, from where the file lives, whether it came with the system or was added to it afterwards.

Exercise 2: Enumerate listening sockets.

```
ss -tlnp                        # TCP sockets in LISTEN state, with owning process
sudo ss -tlnpe                  # -e adds extended info, including the owning uid
```

2a. List every TCP port in the LISTEN state, and for each: the address it is bound to (`127.0.0.1` versus `0.0.0.0` matters, explain the difference), the owning program, and the **user** behind it. `ss -tlnp` shows the process name and PID but not the user; get the user either from the `uid:` field that `ss -e` adds, or by looking the PID up with `ps -o user,pid,cmd -p <PID>`.

2b. One service listens on port **8081**. Identify the process, the user it runs as, and where its unit file lives:

```
sudo ss -tlnp | grep 8081
systemctl status report.service
systemctl cat report.service
```

Record the user the service runs as. Is it `root`, or something else? In one or two sentences, note *why an operator might deliberately run a network service as a non-root user*, you are describing a defensive choice, not a flaw.

2c. Fetch what the service actually serves, and note anything of interest for later:

```
curl -s http://127.0.0.1:8081/ | head -40
```

Exercise 2d: What is the SSH posture, really? Port 22 is the other thing listening, and unlike the report service it is one you will meet again in Lab 5 and in the capstone. Its configuration is assembled from a main file plus a directory of drop-ins, which is where reading carelessly goes wrong.

```
ls /etc/ssh/sshd_config.d/
sudo grep -r PasswordAuthentication /etc/ssh/sshd_config.d/ /etc/ssh/sshd_config
grep -n '^Include' /etc/ssh/sshd_config
```

You will find **two drop-ins that contradict each other** on whether password authentication is allowed. Before running anything else, **write down which one you think wins, and why**. Then ask the server itself, which is the only answer that counts:

```
sudo sshd -T | grep -iE '^passwordauthentica-
↪  tion|^pubkeyauthentication|^permitrootlogin|^authenticationmethods|^trustedusercakeys|^aut
```

2d-i. State the effective setting and whether your prediction was right. Then explain the rule that produces it: the `Include` sits near the top of `sshd_config`, the drop-ins are read in **lexical filename order**, and for each keyword `sshd` keeps the **first** value it sees, not the last. Say which of the two files is therefore dead weight.

2d-i-bis. **Most of those six keywords are printed although nobody wrote them.** For each of the six, `grep -ri` under `/etc/ssh/` and say which are written in a file and which the daemon supplied by itself. Do not take the answer from this handout: run it, because the point of the exercise is that the file and the daemon are different sources and only one of them is the posture.

The lesson is narrower than "defaults are dangerous" and stronger than it. The absence of a keyword from every file does not mean no decision was taken: it means the decision was taken somewhere you did not look. Exercise 2d has just shown you a harmful value that *was* written, in a drop-in nobody opened; this shows you the other half, values that were never written at all. Auditing effective configuration means both.

Then take `authenticationmethods`, whatever your machine prints for it, and say what that value admits that `publickey` would not. You will write `AuthenticationMethods publickey` yourself in Lab 5, Part C, and this is the only place in the unit you meet the value it replaces.

2d-ii. This is an enumeration lesson before it is an SSH lesson. A defender who opened only one of those two files would have reported the opposite posture in good faith. In one or two sentences, say what that tells you about auditing a service whose configuration is assembled from fragments, and what you would do instead. Keep this answer: **Lab 5 Exercise 6b asks you to explain why exactly this default re-opens an initial-access path that a hardened configuration had closed**, and the capstone target ships with it on.

Part C: What runs with privilege? Setuid binaries and sudo

A program that runs with more privilege than the user who launches it is a classic escalation surface. There are two questions: which binaries carry the setuid bit, and what may `student` do through `sudo`.

Exercise 3: Setuid binaries.

```
find / -perm -4000 -type f 2>/dev/null
```

3a. Record the full list. Most entries (`/usr/bin/sudo`, `/usr/bin/passwd`, `mount`, `su`, ...) are standard system binaries present on every Ubuntu machine. **One entry is not standard**: a binary under `/usr/local/`. Name it.

3b. For that non-standard binary, record its owner, permissions and size, and see what it does when run:

```
ls -l /usr/local/bin/backup-tool
/usr/local/bin/backup-tool --version 2>&1 | head -1   # what is this tool, really?
/usr/local/bin/backup-tool 2>&1 | head
```

Note what `--version` reveals: `backup-tool` is not a bespoke utility at all but a copy of a standard system program, given the setuid bit. Do **not** attempt to exploit it. Setuid semantics, real, effective and saved user IDs, are taught in Block 3 (Lab 6). For now, write down a single *question* you would want answered about this binary before deciding whether it is safe. (A good question here is worth more than a wrong answer.)

Exercise 4: Sudo rights.

```
sudo -l
```

4a. What is `student` permitted to run through `sudo`? You will find that `student` is a **full administrator** of this machine, which is expected: it is *your* VM and you own it. Record this fact plainly. The interesting question for Lab 2 is a different one: what could an account that is **not** the owner do, the `svc-report` service account you found in Part B, or any unprivileged foothold an attacker might land? The threat model in Lab 2 is built from that attacker's position, not the owner's, so `student`'s own administrative rights are *out of scope* as an attack path.

Part D: What acts on a schedule? Cron and writable paths

Code that runs automatically, especially as `root`, is surface that an attacker does not even need to trigger.

Exercise 5: Scheduled jobs.

```
ls -l /etc/cron.d/ /etc/cron.daily/ 2>/dev/null
cat /etc/cron.d/* 2>/dev/null
```

5a. One job in `/etc/cron.d/` runs as `root`. Record the schedule, the user, and the exact command it executes.

5b. Inspect the script that job runs:

```
ls -l /opt/maint/rotate.sh
cat /opt/maint/rotate.sh
```

Record the file's permissions. Look carefully at the permission string (`-rwx...`). Who, besides `root`, is able to *write* to this file? State it plainly. Then, in one sentence, describe the general shape of the problem created by the combination of who runs this script and who may change it, without yet reaching for exploit mechanics.

Part E: What is exposed at rest? File permissions and secrets

The last dimension is data: configuration and credentials sitting on disk, and who can read them.

Exercise 6: Readable configuration.

```
ls -l /etc/report/
cat /etc/report/report.conf
```

6a. Record the permissions of `/etc/report/report.conf`. Who can read it? What does it contain?

6b. The file contains credentials. (They are deliberately fake for this exercise, and the file says so, but treat them as if real.) In one or two sentences: what is the difference, from an attacker's point of view, between a secret that only `root` can read and one that *every* user on the machine can read? You are describing the value of a file permission as a security control.

Part F: Consolidate the attack surface inventory

Exercise 7. Bring your findings together into a single table in your notes. Use **one row for each of the four focus items** below; your full enumeration from Parts B to E (all listeners, the standard setuid binaries, the sudo rights) stays in your notes as supporting evidence, and the table distils it to the items that will drive Lab 2:

#	Surface item	Dimension	Runs/owned as	Who can reach/modify it	Open question

Fill one row for each of: the port-8081 service, the non-standard setuid binary, the root cron job and its script, and the readable credentials file. The **Open question** column is where you record what you would need to know (a mechanism from a later week) to judge whether the item is actually dangerous. This table is **the deliverable that feeds Lab 2**; keep it.

Reflection

In a short piece (150 to 250 words), answer:

1. You enumerated four *dimensions* of local attack surface: what listens, what is privileged, what is scheduled, and what is exposed at rest. Why is it useful to think in dimensions rather than as one undifferentiated list of ”problems”?
2. This lab asked you repeatedly to record an observation and *defer* the judgement. What is the risk, for a security practitioner, of collapsing observation and judgement into one step, of declaring something a vulnerability the moment you see it?

References

- `ss(8)`, https://man7.org/linux/man-pages/man8/ss.8.html
- `find(1)`, the `-perm` primary, https://man7.org/linux/man-pages/man1/find.1.html
- setuid / setgid overview, https://man7.org/linux/man-pages/man7/credentials.7.html
- `crontab(5)`, https://man7.org/linux/man-pages/man5/crontab.5.html
- W. Du, *Computer Security: A Hands-on Approach*, 3rd ed., 2022: Ch. 1 (set-UID programs).

Deliverables

Answer every question in this handout. Lab 1 has no report of its own, and **two** things from it are marked, both inside **Lab report 1**, which you write in Lab 2 and submit on Blackboard by **Sunday 11 October**, one week after this block ends:

- the attack-surface inventory table (Exercise 7), as an appendix;

- your answers to the preparation questions P1 and P2, where the P1 capture is also your proof that the machine was running before the session.

The rest carry no mark of their own, and they are still the work this block is built on: Lab report 1 is a threat model of *this* machine, Lab 2 starts from what you recorded today, and a threat model of a generic Linux box is marked at band 1 of 3 for technical correctness. Bring your notes and the completed inventory to the Lab 2 session.

The exact time is the one on the Blackboard assignment for Lab report 1.